

Machine Learning Project Documentation

Assignment 3

Model Refinement, Test Submission & Deployment

PART A: Model Refinement

1. Overview of Refinement Strategy

The refinement phase focuses on building a robust machine learning pipeline using a real-world EPA New York water quality dataset. The main objective is to predict water potability labels derived from World Health Organization (WHO) guidelines based on key chemical indicators such as pH and sulfate concentration.

The dataset was transformed into a structured format and a binary classification target was created using the following WHO-based rule:

- pH between 6.5 and 8.5
- Sulfate ≤ 250 mg/L

To improve model robustness and simulate real-world sensor uncertainty, a 12% label noise was intentionally introduced into the dataset. This forces the models to learn generalized patterns rather than memorizing deterministic rules.

Additionally, all models were trained using a consistent preprocessing pipeline, ensuring reproducibility and eliminating inconsistencies between training stages.

2. Model Evaluation (Baseline Analysis)

A Logistic Regression model was initially implemented as a baseline classifier to evaluate the separability of the dataset.

The baseline model achieved:

- Accuracy: 0.850
- Precision: 0.795
- Recall: 0.823
- F1-score: 0.809

- ROC-AUC: 0.893

Although these results indicate strong predictive capability, error analysis shows that misclassifications still exist, particularly in cases close to decision boundaries. In high-stakes applications such as public health monitoring, even small classification errors may be significant. This observation motivated the use of more advanced non-linear and ensemble-based models.

3. Refinement Techniques

To improve predictive performance, multiple advanced machine learning algorithms were introduced and evaluated:

- Random Forest (ensemble learning)
- XGBoost (gradient boosting framework)
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Multi-Layer Perceptron (MLP neural network)

These models were selected to represent a diverse set of learning paradigms, including:

- Tree-based ensembles (Random Forest, XGBoost)
- Margin-based classification (SVM)
- Distance-based learning (KNN)
- Neural network-based learning (MLP)

A unified pipeline structure was applied across all models using StandardScaler and consistent train-test splits to ensure fair comparison.

4. Hyperparameter Tuning

Hyperparameter optimization was performed using RandomizedSearchCV with 5-fold cross-validation.

Each model was tuned using relevant parameter grids:

- Random Forest: number of estimators, tree depth, and split criteria
- XGBoost: learning rate, depth, and number of estimators
- SVM: kernel type and regularization parameter (C)
- KNN: number of neighbors and weighting strategy
- MLP: hidden layer structure, activation function, and

regularization term

The optimization process used recall as the primary scoring metric. This decision was made to prioritize the detection of non-potable water samples, minimizing the risk of false negatives, which are critical in water safety applications.

5. Cross-Validation

Model evaluation during hyperparameter tuning was performed using 5-fold cross-validation through RandomizedSearchCV.

Since the problem is a classification task, the cross-validation process preserves class distributions across folds using stratified sampling internally. This ensures that each fold contains a representative distribution of potable and non-potable samples.

This approach reduces variance in performance estimation and ensures that reported results reflect true generalization performance rather than dependence on a specific train-test split.

6. Feature Selection

The dataset was intentionally reduced to two key chemical indicators:

- pH
- Sulfate

These features were selected based on WHO water quality guidelines, which define acceptable thresholds for safe drinking water.

Rather than performing traditional feature selection techniques, domain knowledge was used to construct a simplified and interpretable feature space.

To further analyze model behavior, feature importance was extracted from the optimized Random Forest model. The results confirmed that both features contribute significantly to classification decisions, supporting their validity as primary predictive variables.

By limiting the feature space to the most relevant chemical indicators, the model remains:

- Highly interpretable

- Computationally efficient
- Less sensitive to noise

7. Model Performance Summary

All models were evaluated using a consistent test set with the following results:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.850	0.795	0.823	0.809	0.893
Random Forest	0.896	0.911	0.808	0.856	0.876
XGBoost	0.888	0.896	0.803	0.847	0.890
SVM	0.793	0.684	0.862	0.763	0.893
KNN	0.865	0.840	0.803	0.821	0.877
MLP	0.882	0.873	0.813	0.842	0.888

Among all models, Random Forest achieved the best overall balance between accuracy and precision, while SVM achieved the highest recall.

Conclusion

The refinement phase demonstrated that machine learning models can effectively learn WHO-based water potability rules from chemical indicators. Ensemble-based models, particularly Random Forest and XGBoost, provided the most stable and accurate performance.

Although all models achieved strong results, Random Forest was selected as the most suitable candidate for further evaluation and deployment due to its superior balance between accuracy, precision, and robustness against noise.

PART B: Test Submission

1. Overview

The test submission phase represents the final stage of the machine learning workflow, where the refined and

optimized models are evaluated on previously unseen data. The primary objective of this phase is to assess the generalization capability of the trained models and determine their suitability for real-world water potability prediction tasks.

The workflow began with the preparation of the raw water quality dataset obtained from the EPA New York water monitoring records. After data cleaning, feature extraction, and target variable creation, the dataset was divided into training and testing subsets. Several machine learning algorithms, including Logistic Regression, Random Forest, XGBoost, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Multi-Layer Perceptron (MLP), were trained and optimized using hyperparameter tuning. The best-performing models were then applied to the test dataset to generate predictions and evaluate performance using multiple classification metrics.

The overall pipeline can be summarized as follows:

Raw Dataset → Data Cleaning → Feature Engineering → Train/Test Split → Model Training → Hyperparameter Optimization → Test Prediction → Performance Evaluation

2. Data Preparation for Testing

Before evaluating the models, the test dataset underwent the same preprocessing procedures applied during model training to ensure consistency and prevent data leakage.

The following preprocessing steps were performed:

1. Removal of missing values in the selected water quality measurements.
2. Conversion of measurement values into numerical format.
3. Transformation of the dataset into a structured wide-format representation using pivot tables.
4. Creation of the target variable (*Potability*) based on WHO drinking water guidelines:
 - pH between 6.5 and 8.5
 - Sulfate concentration below 250 mg/L
5. Selection of the final predictor variables:

- pH
- Sulfate

The dataset was then divided into training and testing subsets using an 80:20 ratio with stratified sampling to preserve class distribution.

Unlike the training data, no oversampling techniques such as SMOTE were applied directly to the test dataset. SMOTE was only applied within the training pipeline to avoid information leakage and maintain an unbiased evaluation process.

Furthermore, feature scaling parameters were learned exclusively from the training data and then applied to the test data using the fitted scaler.

3. Model Application

After training and optimizing each model, the trained estimators were applied to the test dataset to generate predictions and probability scores.

The prediction process consisted of:

1. Applying the same preprocessing pipeline used during training.
2. Transforming test features using the fitted scaler.
3. Feeding the processed data into the trained model.
4. Generating class predictions and prediction probabilities.
5. Computing evaluation metrics.

```
# Scale test features using the scaler fitted on training data
X_test_scaled = scaler.transform(X_test)

# Generate class predictions
y_pred = lr.predict(X_test_scaled)

# Generate probability estimates
y_prob = lr.predict_proba(X_test_scaled)[:, 1]

# Evaluate model performance
evaluate_model(
    "Baseline Logistic Regression",
    y_test,
    y_pred,
    y_prob
)
```

```
# Predict labels using optimized Random Forest
rf_predictions = best_rf.predict(X_test)

# Predict probabilities
rf_probabilities = best_rf.predict_proba(X_test)[:, 1]

# Evaluate model performance
evaluate_model(
    "Optimized Random Forest",
    y_test,
    rf_predictions,
    rf_probabilities
)
```

The same prediction procedure was applied to XGBoost, SVM, KNN, and MLP models.

4. Test Metrics

To evaluate model performance comprehensively, several classification metrics were used:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC Score

Test Set Results

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.850	0.795	0.823	0.809	0.893
Random Forest	0.896	0.911	0.808	0.856	0.876
XGBoost	0.888	0.896	0.803	0.847	0.890
SVM	0.793	0.684	0.862	0.763	0.893
KNN	0.865	0.840	0.803	0.821	0.877
MLP	0.882	0.873	0.813	0.842	0.888

Comparison with Training and Validation Performance

The test performance remained consistent with the cross-validation results obtained during hyperparameter optimization, indicating that the models generalized

effectively to unseen data.

Random Forest achieved the highest overall accuracy (89.6%) and F1-score (85.6%), making it the strongest overall classifier. XGBoost and MLP also demonstrated competitive performance with balanced precision and recall values.

SVM achieved the highest recall (86.2%), indicating superior sensitivity in identifying potable water samples. However, its lower precision suggests a higher number of false positive classifications.

The baseline Logistic Regression model achieved surprisingly strong results despite its simplicity, indicating that the selected features contain substantial predictive information.

No severe signs of overfitting were observed, as the test performance remained close to the performance observed during model optimization. The introduction of 12% label noise further contributed to creating a more realistic and robust evaluation scenario.

5. Model Deployment

In this project, the primary objective was model development and performance evaluation rather than full-scale deployment. Therefore, the models were executed within a local Python environment using Jupyter Notebook and standard machine learning libraries.

Deployment Environment

- Local machine environment
- Python 3.x
- Scikit-learn
- XGBoost
- Imbalanced-learn
- Pandas and NumPy

Although not implemented in this study, the optimized Random Forest model could be deployed as a REST API using frameworks such as:

- Flask
- FastAPI
- Docker containers

- Cloud services (AWS, Azure, Google Cloud Platform)

Such deployment would allow real-time water quality measurements to be submitted and classified automatically.

No external APIs or third-party systems were integrated during this project.

6. Code Implementation

Model Refinement

The following code demonstrates the refinement and optimization of the Random Forest model using hyperparameter tuning and SMOTE-based class balancing.

```
rf_pipeline = ImbPipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(random_state=42))
])

rf_params = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [None, 5, 10],
    'classifier__min_samples_split': [2, 5]
}

rf_search = RandomizedSearchCV(
    rf_pipeline,
    rf_params,
    n_iter=5,
    cv=5,
    scoring='recall',
    random_state=42,
    n_jobs=-1
)

rf_search.fit(X_train, y_train)

best_rf = rf_search.best_estimator_
```

Test Submission

The following code illustrates how the optimized model was evaluated on the test dataset.

```
# Generate predictions
y_pred = best_rf.predict(X_test)

# Generate probability scores
y_prob = best_rf.predict_proba(X_test)[:, 1]

# Calculate evaluation metrics
evaluate_model(
    "Optimized Random Forest",
    y_test,
    y_pred,
    y_prob
)
```

This procedure was repeated for all optimized models to facilitate comparative performance analysis.

Conclusion

The model refinement and test submission phases successfully demonstrated the effectiveness of machine learning techniques for water potability prediction using EPA New York water quality data.

Multiple classification algorithms were trained and optimized using hyperparameter tuning and imbalance-handling techniques. Among all evaluated models,

Random Forest achieved the best overall performance, obtaining an accuracy of 89.6%, precision of 91.1%, and F1-score of 85.6%. XGBoost and MLP also produced strong results, while SVM achieved the highest recall, making it particularly effective for detecting potable water samples.

One challenge encountered during the project was preventing overfitting while maintaining realistic performance expectations. To address this issue, a 12% label-noise injection strategy was introduced, simulating real-world measurement and calibration errors. Despite this added complexity, the models maintained strong predictive performance.

Overall, the results indicate that water potability can be predicted effectively using only pH and sulfate measurements, and the optimized machine learning models demonstrate strong potential for future integration into automated water quality monitoring systems.

PART C: Deployment

1. Overview

The deployment phase focuses on making the trained machine learning model accessible for real-world use. After evaluating multiple classification algorithms, the optimized Random Forest model was selected as the primary deployment candidate due to its superior overall performance in terms of accuracy, precision, and F1-score.

The main goal of deployment is to provide an automated system capable of predicting water potability based on chemical measurements such as pH and sulfate concentration. Such a system can support water quality monitoring agencies, environmental researchers, and decision-makers by providing fast and reliable predictions.

The proposed deployment strategy consists of exposing the trained model through a lightweight web API that receives water quality measurements as input and returns potability predictions in real time.

Target Environment

The deployment architecture is designed for a cloud-based environment due to its scalability and accessibility. However, the solution can also be executed in:

- Cloud platforms (AWS, Microsoft Azure, Google Cloud Platform)
- On-premises servers
- Edge devices used in water monitoring stations

For this project, deployment is assumed to be performed on a cloud-hosted Python environment using a RESTful API.

2. Model Serialization

Before deployment, the trained machine learning model must be serialized into a format that can be stored and loaded efficiently without retraining.

For this project, the Random Forest model can be serialized using the Python Pickle format.

Serialization Format

The following formats are commonly used:

Format	Purpose
Pickle (.pkl)	Native Python model serialization
Joblib (.joblib)	Efficient storage of large models
ONNX	Cross-platform deployment
TensorFlow SavedModel	TensorFlow model deployment

Since the project is based primarily on Scikit-learn, Pickle or Joblib are the most suitable options.

Example Serialization Code

```
import joblib

# Save trained model
joblib.dump(best_rf, "water_potability_model.joblib")

# Load model later
model = joblib.load("water_potability_model.joblib")
```

Storage and Versioning

To ensure reproducibility and maintainability, each model version should be stored with:

- Model version number
- Training date
- Dataset version
- Hyperparameter configuration
- Performance metrics

Example naming convention:

```
water_model_v1.0.joblib  
water_model_v1.1.joblib  
water_model_v2.0.joblib
```

Version control helps track improvements and facilitates rollback if performance degradation occurs.

3. Model Serving

Once serialized, the model must be served so that external applications can submit requests and receive predictions.

Serving Framework

The proposed deployment uses FastAPI because of its:

- High performance
- Automatic API documentation
- Easy integration with Python machine learning models
- Low latency response times

Example FastAPI Service

```

from fastapi import FastAPI
import joblib
import pandas as pd

app = FastAPI()

model = joblib.load("water_potability_model.joblib")

@app.post("/predict")
def predict(ph: float, sulfate: float):

    data = pd.DataFrame({
        "pH": [ph],
        "Sulfate": [sulfate]
    })

    prediction = model.predict(data)[0]

    return {"potability": int(prediction)}

```

Platform Selection

The deployment service can be hosted on:

- AWS EC2
- AWS Elastic Beanstalk
- Azure App Service
- Google Cloud Run
- Docker containers

Scalability and Latency

Since the model uses only two numerical features and Random Forest inference is computationally lightweight, prediction latency is expected to remain below a few milliseconds per request.

Scalability can be improved through:

- Container orchestration (Docker + Kubernetes)
- Load balancing
- Horizontal scaling

4. API Integration

The deployed model can be accessed through REST API endpoints.

API Endpoints

Endpoint	Method	Purpose
/predict	POST	Generate water potability prediction
/health	GET	Verify service availability
/metrics	GET	Retrieve monitoring metrics

Input Format

Example JSON request:

```
{
  "pH": 7.2,
  "Sulfate": 180
}
```

Output Format

Example JSON response:

```
{
  "potability": 1,
  "label": "Potable Water"
}
```

Example Request

```
POST /predict HTTP/1.1
Content-Type: application/json

{
  "pH": 7.2,
  "Sulfate": 180
}
```

Example Response

```
HTTP/1.1 200 OK

{
  "potability": 1,
  "label": "Potable Water"
}
```

The API enables seamless integration with web applications, mobile applications, environmental monitoring dashboards, and IoT-based water quality systems.

5. Security Considerations

Security is a critical component of any deployed machine learning service, particularly when handling environmental monitoring data.

Authentication and Authorization

The proposed deployment includes:

- API Key authentication
- Role-based access control (RBAC)
- User authorization for administrative endpoints

Example API Key validation:

```
Authorization: Bearer <API_KEY>
```

Data Encryption

To protect sensitive information:

In-Transit Encryption

- HTTPS/TLS encryption
- Secure API communication

At-Rest Encryption

- Encrypted storage volumes
- Encrypted model files and logs

Threat Mitigation

The deployment architecture should include:

- Input validation and sanitization
- Rate limiting
- Firewall protection
- DDoS protection
- Dependency vulnerability scanning

These measures reduce the risk of malicious requests and unauthorized access.

6. Monitoring and Logging

Continuous monitoring ensures that the deployed model remains reliable and accurate over time.

Production Metrics

The following metrics should be monitored:

System Metrics

- API response latency
- Request throughput

- CPU utilization
- Memory usage
- Error rate
-

Model Metrics

- Prediction distribution
- Data drift
- Concept drift
- Accuracy degradation
- False positive rate
- False negative rate

Logging Framework

The deployment service should log:

- Incoming requests
- Prediction results
- Processing time
- API errors
- Security events

Example log entry:

```
2026-06-02 14:25:17
Request ID: 5421
pH: 7.1
Sulfate: 195
Prediction: Potable
Latency: 8 ms
```

Alerting and Anomaly Detection

Automated alerts can be configured when:

- Response latency exceeds predefined thresholds.
- Prediction distributions change significantly.
- API error rates increase.
- Data drift is detected.

Monitoring tools such as Prometheus and Grafana can be integrated to visualize performance metrics and generate real-time alerts.

Conclusion

The deployment phase focused on preparing the optimized Random Forest model for real-world usage

through a scalable and secure API-based architecture. The model was serialized using Joblib, enabling efficient storage and retrieval without retraining. A FastAPI-based serving framework was proposed to provide low-latency prediction services and facilitate integration with external systems.

Security measures such as API authentication, encrypted communication, and input validation were incorporated to ensure reliable operation. Additionally, monitoring and logging mechanisms were proposed to track system performance, detect model drift, and maintain long-term prediction quality.

Overall, the deployment architecture demonstrates how the developed water potability prediction model can be transitioned from an experimental environment into a practical decision-support system for water quality assessment and environmental monitoring applications.